



به نام خدا



دانشگاه تهران
دانشکده مهندسی برق و کامپیوتر
مقدمه ای بر علوم داده
فاز دوم پروژه اصلی

نام و نام خانوادگی	محمد عسکری
شماره دانشجویی	۸۱۰۱۹۸۴۴۱
تاریخ ارسال گزارش	۱۴۰۴/۰۲/۱۱

1. Project Overview

[This project](#) is the second phase of a data science pipeline focused on optimizing a portfolio of S&P 500 stocks over five years using **Markowitz Modern Portfolio Theory**. Phase 2 builds on exploratory work completed in Phase 1 and introduces:

- Structured databases (SQLite)
- Feature engineering
- Modular AI pipeline scripting
- Automation with CI/CD (GitHub Actions)
- Containerization via Docker (Bonus)

2. Dataset Description

- **Source:** `all_stocks_5yr.csv`
- **Fields:**
 - `date`: Trading date
 - `open, high, low, close`: Daily price metrics
 - `volume`: Shares traded
 - `Name`: Stock ticker (e.g., AAPL, MSFT)
- **Size:** ~2 million rows covering 500+ stocks from 2013–2018

3. Section 1: Database Design and SQL Integration

Database: SQLite

- Chosen for simplicity and local compatibility with WSL

Schema:

Table: `stock_prices`

Column	Type	Description
<code>date</code>	DATE	Trading day
<code>open</code>	REAL	Opening price
<code>high</code>	REAL	Daily high
<code>low</code>	REAL	Daily low
<code>close</code>	REAL	Closing price
<code>volume</code>	INTEGER	Daily volume

Name	TEXT	Stock ticker
------	------	--------------

SQL Integration:

- Data loaded into SQLite using Pandas and SQLAlchemy
- Database script: `load_data.py`

Sample Queries Executed:

1. Average return by ticker
2. Volatility per stock
3. Top 5 most traded stocks
4. Daily volume trends
5. Join with future metadata for sector analysis (planned)

```
(.venv) m4skari@LAPTOP-06VNC6E0:~/portfolio_project$ python scripts/load_data.py
Data loaded into 'stock_prices' table in stock_data.db
```

4. Section 2: Feature Engineering & Preprocessing

Preprocessing Tasks:

- Converted `date` to datetime object
- Sorted by `Name` and `date`
- Removed nulls and duplicates

Engineered Features:

Feature	Formula or Method
<code>daily_return</code>	<code>pct_change</code> by ticker
<code>log_return</code>	<code>log(close / prev_close)</code>
<code>volatility_20d</code>	20-day rolling std dev
<code>mean_return_20d</code>	20-day rolling average

- Handled NaNs resulting from rolling operations
- Saved to `features.csv`

```
(.venv) m4skari@LAPTOP-06VNC6E0:~/portfolio_project$ head -n 5 features.csv
date,open,high,low,close,volume,Name,daily_return,log_return,volatility_20d,mean_return_20d
2013-03-11,42.99,43.01,41.73,42.81,5472837,A,-0.005112712061352553,-0.005125826693653537,0.015878726378318643,-0.0024581917800956645
2013-03-12,42.8,43.0,42.59,42.62,3767613,A,-0.004438215370240695,-0.004448093486383757,0.015770507125759625,-0.002147715680817108
2013-03-13,42.56,43.05,42.4,43.0,2241086,A,0.008916001877053104,0.008876489023091419,0.01595643722279536,-0.001724337111628127
2013-03-14,43.03,43.38,43.0,43.32,2840603,A,0.007441860465116301,0.007414306439227461,0.016057518822121704,-0.0014979186737600347
```

5. Section 3: AI Pipeline Design

Folder Structure:

```
portfolio_project/  
├── pipeline.py  
├── requirements.txt  
├── README.md  
├── all_stocks_5yr.csv  
├── cleaned_data.csv  
├── features.csv  
├── stock_data.db  
├── scripts/  
│   ├── database_connection.py  
│   ├── load_data.py  
│   ├── preprocess.py  
│   └── feature_engineering.py
```

Pipeline Components:

- `database_connection.py`: Sets up SQLite connection
 - `load_data.py`: Loads CSV into SQLite
 - `preprocess.py`: Cleans and sorts data
 - `feature_engineering.py`: Generates all required features
 - `pipeline.py`: Calls all scripts in sequence
-

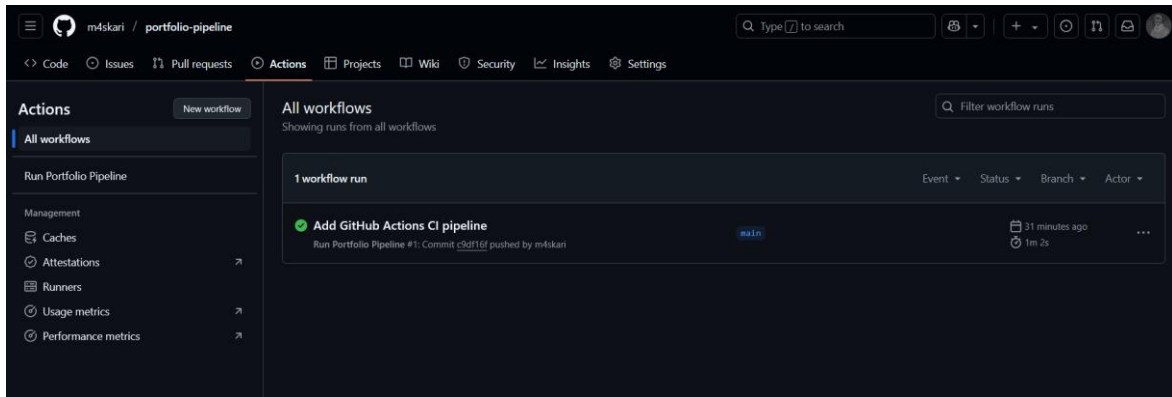
6. Section 4: CI/CD with GitHub Actions

Workflow File: `.github/workflows/pipeline.yml`

- Triggers on push to `main`
- Steps:
 1. Checkout repo
 2. Set up Python 3.12
 3. Install dependencies
 4. Run `pipeline.py`

Result:

- Verified in Actions tab
- Ensures reproducibility and testability on every push



7. Section 5 (Bonus): Docker Integration

Dockerfile:

```
FROM python:3.12-slim
WORKDIR /app
COPY . .
RUN pip install --no-cache-dir -r requirements.txt
CMD ["python", "pipeline.py"]
```

Commands Used:

```
docker build -t portfolio-pipeline .
docker run --rm portfolio-pipeline
```

Output:

- Pipeline executed end-to-end in container
- Verified with screenshot for submission

```
(.venv) m4skari@LAPTOP-06VNC6E0:~/portfolio_project$ docker run --rm portfol
io-pipeline
Data loaded into 'stock_prices' table in stock_data.db
Preprocessing complete. Cleaned data saved to 'cleaned_data.csv'
✅ Feature engineering complete. Saved to 'features.csv'
📦 Loading data into database...

🌸 Preprocessing data...

🧠 Performing feature engineering...

✅ Pipeline execution complete.
```

8. Conclusion

This project implements a complete data science pipeline that aligns with real-world data engineering practices, CI/CD standards, and deployable AI workflows. The combination of reproducibility (via GitHub Actions) and portability (via Docker) strengthens its value as an academic and practical deliverable.